
Sub-Token Routing in LoRA for Adaptation and Query-Aware KV Compression

Wei Jiang

Futurewei Technologies Inc.
San Jose, CA 95131
wjjiang@futurewei.com

Wei Wang

Futurewei Technologies Inc.
San Jose, CA 95131
rickweiwang@futurewei.com

Abstract

Sub-token routing offers a finer control axis for transformer efficiency than the coarse units used in most prior work, such as tokens, pages, heads, or layers. In this paper, we study routing within a token representation itself in LoRA-adapted transformers. The motivation is that a relevant token need not be internally uniform: under a retention budget, preserved value groups are distributed unevenly both across tokens and within tokens, which suggests that KV compression need not be an all-or-nothing decision at token level. We study this fine-grained routing mechanism in two settings. For compression-aware language modeling, we introduce a query-independent design that combines routed subspace LoRA with value-group routing on the KV path. For downstream-task-preserving KV compression, we introduce a query-aware design in which a predictor-based selector allocates a global retention budget over context-token/value-group pairs using query-conditioned relevance. Experiments show that the query-independent design improves the quality-compression tradeoff for language modeling, while the query-aware design preserves downstream behavior under reduced KV budgets. We further examine the relation between token-level and sub-token-level query-aware routing, and show that they form complementary compression axes: token-level methods determine which tokens survive globally, while sub-token routing determines how the surviving tokens are compressed internally.

1 Introduction

Recent work on efficient transformer inference usually acts on coarse structural units such as whole tokens, pages, heads, layers, or tensors. This has led to effective methods for reducing KV-cache cost through token eviction, page selection, head sharing, and low-precision compression [1, 8, 9, 12, 17, 21, 23]. A common feature of these approaches is that the routing or compression decision is made at the level of an entire token representation or another coarse block.

In this paper, we study a finer control axis: routing within a token representation itself. The basic intuition is that a relevant token need not be internally uniform: not every part of its value representation is equally useful. As shown in Figure 1a, under a fixed total retention budget, the number of preserved value groups per token is highly non-uniform: many tokens are fully dropped, some are fully preserved, and many are only partially preserved. Figure 1b further shows that after tokens are ordered by query-conditioned relevance, the number of preserved groups increases systematically with token importance. These patterns suggest that KV compression need not be an all-or-nothing decision at token level. Instead, a token may be worth preserving only partially.

This finer control is especially natural on the value path. In attention, the query-key interaction determines where information is retrieved from, while the value representation carries the retrieved content [19]. If relevance is determined by the query-key geometry, it does not follow that every

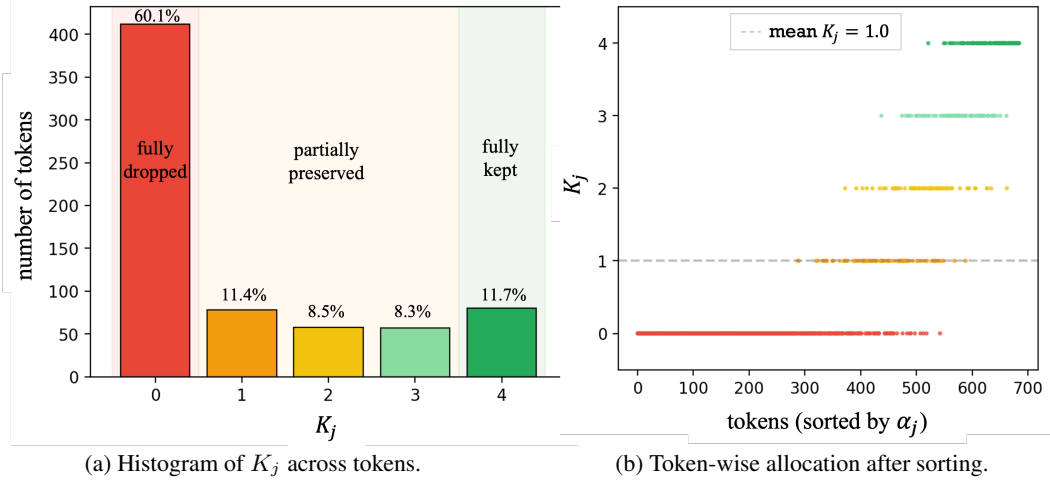


Figure 1: Diagnostics for sub-token imbalance under a fixed total retention budget. Here $K_j \in \{0, \dots, S\}$ denotes the number of preserved value groups for token j , with $S = 4$. (a) The allocation is highly non-uniform: many tokens are fully dropped, some are fully preserved, and many are only partially preserved. (b) Token-wise allocation after sorting tokens by query-conditioned relevance α_j . Although the average budget is fixed at $\mathbb{E}[K_j] = 1$, higher-relevance tokens systematically receive larger internal allocation. These diagnostics show that preserved information is not only sparse across tokens but also unevenly distributed within tokens, which motivates routing at the level of internal value groups rather than only at token level.

internal group of the corresponding value vector must contribute equally. Therefore a token may be worth preserving only partially. This motivates routing at the level of internal value groups, which we refer to as *sub-token routing*.

We study this finer sub-token routing in LoRA-adapted transformers [6], where routing can be introduced both on the adaptation path and on the KV path. We consider two settings. The first is *query-independent routing*, where no downstream query is available and routing is learned from the token representation itself. This setting is relevant to adaptation and compression-aware language modeling. The second is *query-aware routing*, where the routing decision depends on the current query and the goal is downstream-task-preserving KV compression. This second setting is related to recent query-aware token-level selection methods such as Quest and Expected Attention [2, 18], but our routed object is finer, namely internal value groups within each token representation.

Our query-independent design combines routed subspace LoRA with value-group routing on the KV path. This gives a model that improves language-model quality while reducing KV cache. Our query-aware design uses a predictor-based selector that allocates a global retention budget over context-token / value-group pairs using query-conditioned relevance. The resulting method preserves downstream performance at practical KV budgets while operating at the finer granularity of internal value groups.

Sub-token routing is also naturally complementary to token-level selection: token-level methods decide which tokens or pages survive globally [2, 18], while sub-token routing decides how the surviving tokens are compressed internally. Our experiments show that these two can be combined to achieve deeper KV compression at nearly unchanged task accuracy.

Our contribution can be summarized as follows.

- We study fine-grained routing in LoRA-adapted transformers, using routed subspaces for adaptation and routed value groups for KV compression.
- We introduce a query-independent design for compression-aware language modeling that combines routed subspace LoRA with value-group routing on the KV path.
- We introduce a query-aware value-group routing design for downstream-task-preserving KV compression, where a predictor-based selector allocates a global retention budget over context-token / value-group pairs using query-conditioned relevance.

- We show experimentally that the query-independent design improves the quality-compression tradeoff for language modeling, while the query-aware design preserves downstream performance under reduced KV budgets.
- We analyze the relation between token-level and sub-token-level query-aware routing, and show that they provide complementary compression axes whose combination enables deeper KV compression at similar task accuracy.

2 Related Work

LoRA-based adaptation. Parameter-efficient fine-tuning adapts a frozen pretrained model by introducing a small set of trainable parameters rather than updating the full network. LoRA is the most widely used baseline, where the base model is kept frozen while trainable low-rank updates are injected into selected projections [6]. Subsequent variants improve this formulation in various ways. AdaLoRA reallocates the parameter budget adaptively across weight matrices according to estimated importance [22], while DoRA decomposes the pretrained weight into magnitude and direction and applies low-rank adaptation to the directional component [11]. These methods improve adaptation quality or parameter efficiency, but they do not directly address KV-cache cost at inference time.

Routed adaptation with multiple LoRA branches. A separate line of work introduces routing over multiple LoRA modules. MoELoRA treats LoRA adaptation as a mixture of experts and uses token-dependent routing over multiple low-rank branches [14]. Related methods such as MoLE and AdaMoLE also route among multiple LoRA experts or low-rank branches rather than using a single shared update [13, 20]. These methods are the closest adaptation-side baselines to our routed LoRA formulation. The main difference is the routing unit. Prior routed-LoRA methods choose among whole LoRA branches or experts. Our method instead routes among internal subspaces within a projection, which is the same finer level of control that we later use on the KV path.

Efficient transformer inference at coarse structural units. Most existing KV-efficiency methods operate at relatively coarse structural units such as heads, tokens, pages, layers, or tensors. Multi-query attention reduces decode-time memory and bandwidth by sharing keys and values across heads [17], and grouped-query attention shares KV states within head groups rather than globally [1]. Token-level retention methods prune or preserve whole tokens. StreamingLLM combines sink tokens with a recency window to stabilize long-context decoding [21], while H₂O preserves heavy-hitter tokens according to accumulated attention mass [23]. Other approaches reduce KV cost through compression or quantization. CacheGen compresses KV states for memory-efficient generation [12], MiniCache exploits cross-layer redundancy in KV states [9], and residual-vector-quantization-style approaches target reduced-precision KV representations [8]. These methods are effective when the relevant redundancy is visible at the level of tokens, heads, layers, or numerical precision. Our work studies a different control axis: routing within a token’s value representation itself.

Query-aware KV selection. Many KV-retention methods are effectively query-independent at the time the cache is maintained: they rely on recency or past attention statistics rather than the actual downstream query [21, 23]. More recent work moves toward query-aware selection. Quest performs query-conditioned sparse retrieval for long-context inference by selecting query-relevant KV content during attention [18]. Expected Attention estimates future-query attention in order to rank KV entries before the actual downstream attention is observed [2]. Query-aware cache-selection methods in serving systems similarly allocate cache resources according to predicted query relevance rather than generic retention rules [10]. These methods are closely related to our query-aware setting, but they typically act on tokens, pages, or other coarse cache units. Our work addresses the same question with a finer routed object, namely internal groups within the value representation.

Positioning of the present work. Our work lies at the intersection of LoRA-based adaptation and KV-cache compression. On the adaptation side, we study routing inside a LoRA-style update rather than across whole LoRA experts. On the cache side, we study routing inside the value representation rather than across whole tokens, heads, or pages. The same fine-grained control axis is then studied in two settings. In the query-independent setting, it is used for adaptation and compression-aware language modeling. In the query-aware setting, it is used for downstream-task-preserving KV compression. This finer routed object is naturally complementary to token-level selection: token-level

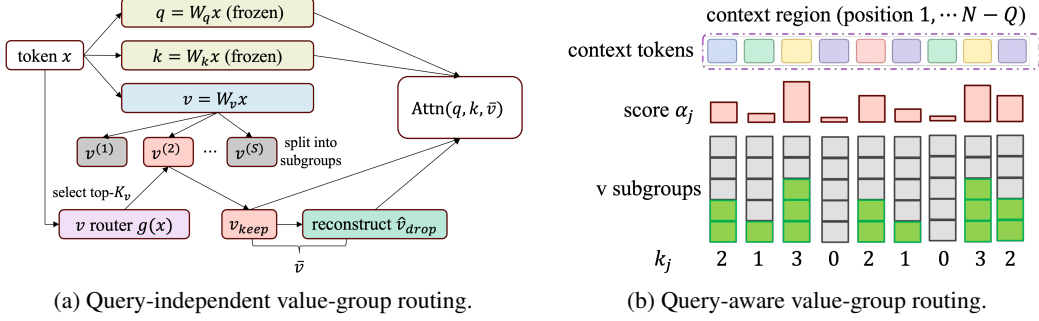


Figure 2: Value-group routing for KV compression. The query and key are kept unchanged, and the value is divided into subgroups. (a) Query-independent routing uses a value router to select the subgroups to preserve, from which the dropped subgroups are reconstructed. (b) Query-aware routing first estimates the relevance score of each context token. Important tokens retain more value groups, while less relevant tokens retain fewer or none.

methods decide which tokens or pages survive globally, while sub-token routing determines how the surviving tokens are compressed internally. We also study this complementarity directly by combining token-level query-aware selection with sub-token value-group routing.

3 Method

We study fine-grained routing in LoRA-adapted transformers. Our focus is the quality-compression tradeoff. The basic routed object is groups within the value representation. In the query-independent setting, routing is learned without access to a downstream query and is used for compression-aware language modeling. In the query-aware setting, routing depends on query-conditioned relevance and is used for downstream-task-preserving KV compression.

3.1 LoRA Adaptation with Query-Independent Routing

We first consider the setting in which no downstream query is available. For LoRA-style adaptation we incorporate fine-grained routing on both the adaptation side and the cache side.

3.1.1 Routed subspace LoRA

Let a frozen projection matrix W be adapted by a LoRA update ΔW , so that

$$y = (W + \Delta W)x, \quad (1)$$

where x is the input token representation. In standard LoRA, ΔW is a single low-rank update shared across all tokens.

We replace this with a routed collection of low-rank subspace updates. Specifically,

$$\Delta W(x) = \sum_{s=1}^S r_s(x) \Delta W^{(s)}, \quad (2)$$

where $\Delta W^{(s)}$ is the low-rank update associated with subspace s , and $r_s(x)$ is a routing weight predicted from the current token representation.

The routing weights are produced by a lightweight token-wise linear router. Given the input token representation $x \in \mathbb{R}^d$ to the adapted projection, the router outputs S routing scores,

$$a(x) = W_r x + b_r \in \mathbb{R}^S. \quad (3)$$

The routing weights $r(x)$ are then obtained by applying sparse top- K selection to these scores, followed by normalization over the selected subspaces. Thus, for each token, only a small subset of the S subspace updates is activated.

The resulting adaptation remains token-dependent, but the routing unit is an internal subspace within a projection rather than a whole expert branch. This differs from expert-routed LoRA methods such as MoE-LoRA. MoE-LoRA routes among whole LoRA branches, whereas routed subspace LoRA routes within a single projection update.

3.1.2 Value-group routing for KV compression

On the cache side, we route groups within the value representation while leaving the query and key paths unchanged. As illustrated in Figure 2 (a), for a self-attention layer with input token representation $x \in \mathbb{R}^{d_{\text{model}}}$, we compute

$$q = W_q x, \quad k = W_k x, \quad v = W_v x,$$

where $v \in \mathbb{R}^{d_{\text{head}}}$. We partition the value vector into S groups,

$$v = [v^{(1)} \mid v^{(2)} \mid \dots \mid v^{(S)}], \quad (4)$$

where each $v^{(s)} \in \mathbb{R}^{d_g}$, $d_g = d_{\text{head}}/S$. A routing function assigns one score to each value group,

$$\alpha = g(x) \in \mathbb{R}^S, \quad (5)$$

and selects the top- K groups. Let $\mathcal{I}_{\text{keep}}$ denote the selected group indices:

$$v_{\text{keep}} = \{v^{(s)} : s \in \mathcal{I}_{\text{keep}}\},$$

v_{keep} is written to cache. The dropped groups v_{drop} are reconstructed from the preserved subset,

$$\hat{v}_{\text{drop}} = R(v_{\text{keep}}), \quad (6)$$

where $R(\cdot)$ is a learned reconstructor. The routed value used by attention is $\tilde{v}^{(s)} = v^{(s)}$ if $s \in \mathcal{I}_{\text{keep}}$, and $\hat{v}_{\text{drop}}^{(s)}$ otherwise. Attention is then computed by

$$\text{Attn}(q, k, \tilde{v}) = \text{softmax}\left(\frac{qk^\top}{\sqrt{d_{\text{head}}}}\right) \tilde{v}, \quad (7)$$

where $\tilde{v} = [\tilde{v}^{(1)} \mid \tilde{v}^{(2)} \mid \dots \mid \tilde{v}^{(S)}]$. Thus, compression is only on the value path. The query-key interaction remains unchanged, while routing controls the content stored in the value cache.

Each selected projection is adapted by routed subspace LoRA, so the low-rank update depends on the current token representation rather than being shared uniformly across tokens. At the same time, the value representation on the KV path is routed at the group level. The resulting model therefore adapts the frozen base model through routed LoRA and reduces KV cost through value-group routing.

3.2 Query-Aware Value-Group Routing

We next consider downstream-task-preserving KV compression. The routed object is still groups within the value representation. The difference is that routing is no longer determined from the token representation alone. Instead, it depends on query-conditioned relevance.

3.2.1 Query-conditioned value-group selection

The input sequence is divided into a context region and a query region, with token index sets $\mathcal{C}$ and $\mathcal{Q}$, respectively. The context region contains the earlier tokens whose KV entries may later be reused. The query region contains the later tokens and determines which part of the context is relevant.

As illustrated in Figure 2 (b), for each context token $j \in \mathcal{C}$, let $v_j^{(g)}$ denote its g -th value group. We assign an importance score to each context-token / value-group pair by

$$s_{j,g} = \alpha_j \cdot \|v_j^{(g)}\|_2, \quad j \in \mathcal{C}, \quad g = 1, \dots, S, \quad (8)$$

where α_j is a query-conditioned relevance signal for context token j , measuring token-level relevance to the current query. The second factor measures the magnitude of the corresponding value group.

Given a retention budget $\rho \in (0, 1]$, we define the total number of preserved context-token / value-group pairs as

$$M = \lfloor \rho |\mathcal{C}| S \rfloor. \quad (9)$$

We then select the top- M pairs (j, g) according to $s_{j,g}$, and define the routing mask $m_{j,g} = 1$ if (j, g) is among the top- M scores. This produces a token-dependent number of preserved groups,

$$K_j = \sum_{g=1}^S m_{j,g}, \quad j \in \mathcal{C}, \quad (10)$$

so the allocation is adaptive. Important context tokens may keep many value groups, while less relevant tokens may keep few or none.

In addition to the global budget, we impose a constraint. The query-region tokens are not compressed, since they define the task and should remain unchanged.

The routed value is then

$$\tilde{v}_j^{(g)} = m_{j,g} v_j^{(g)}, \quad \tilde{v}_j = [\tilde{v}_j^{(1)} \mid \tilde{v}_j^{(2)} \mid \cdots \mid \tilde{v}_j^{(S)}], \quad j \in \mathcal{C}, \quad (11)$$

while the query-region tokens are kept unchanged. Attention is then computed with the original query and key and the routed value, as in Eqn. (7).

3.2.2 Predictor-based query-aware routing

One intuitive way to define the query-conditioned relevance signal is to use a diagnostic attention layer. Let A_{qj} denote the query-to-context attention mass from query token $q \in \mathcal{Q}$ to context token $j \in \mathcal{C}$, averaged over heads, at a designated layer. Then

$$\alpha_j = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} A_{qj}, \quad (12)$$

which, together with Eqn. (8), gives an explicit query-conditioned selector.

This construction is useful conceptually, but it is not practical for deployment, since obtaining α_j in this way requires an additional diagnostic computation before the compressed forward pass. We therefore adopt a predictor-based approach.

Let $h_j^{(L_{\text{split}})}$ denote the hidden state of context token j at a split layer L_{split} . A lightweight predictor produces one score per value group,

$$\hat{s}_{j,:} = \phi\left(h_j^{(L_{\text{split}})}\right), \quad j \in \mathcal{C}, \quad (13)$$

and the same global top- M rule is then applied to $\hat{s}_{j,g}$. In this way, the predictor provides a learned approximation to the query-conditioned selection rule without requiring the explicit diagnostic computation at inference time.

The predictor-based selector induces a depth-wise split in the network. Layers before L_{split} operate on the full context and provide the information used for query-aware selection. Layers after L_{split} operate on the routed value representation defined by the selected token-group pairs. Query-aware value-group routing is therefore carried out in a single forward pass.

3.3 Architecture Details

Query-independent value-group routing. On the cache side, each attention layer has a lightweight linear value-group router that produces S group logits. A hard top- K rule selects groups to keep. Each attention layer also includes a small reconstructor $R(\cdot)$, implemented as a two-layer MLP with GELU activation, which predicts the full value representation from the kept groups. The router and reconstructor are trained together with the routed LoRA modules through the auxiliary cache-side objective described in the Appendix.

Query-aware predictor. The score in Eqn. (13) is predicted by a two-layer MLP with GELU activation, shared across all compressed layers. The predictor is trained to approximate query-conditioned token-group relevance scores, as described in the Appendix. The query region contains the last 16 tokens and is kept uncompressed.

3.4 Optimization

The detailed optimization objectives are given in the Appendix. Briefly, the query-independent design combines the standard language-modeling objective with an auxiliary loss for value-group routing and reconstruction on the cache path, while the query-aware design trains the predictor to approximate query-conditioned token-group relevance scores.

Table 1: Query-independent routing for language modeling across Qwen2.5 model scales. LoRA and MoE-LoRA are full-cache baselines. KV retention percentages are shown in parentheses.

Model	LoRA	MoE-LoRA	Sub-token
Qwen2.5-7B	35.08 (100%)	33.72 (100%)	31.41 (75%)
Qwen2.5-14B	25.42 (100%)	24.49 (100%)	23.43 (75%)
Qwen2.5-32B	27.10 (100%)	25.76 (100%)	23.78 (75%)
Qwen2.5-72B	21.28 (100%)	20.93 (100%)	19.97 (75%)

4 Experiments

4.1 Experimental Setup

Models. Our main experiments use the Qwen 2.5 family [16], including 7B, 14B, 32B, and 72B models. We also include Mistral-7B [7] in the Appendix as cross-family validation.

Methods and baselines. For the query-independent setting, we evaluate routed subspace LoRA with value-group routing, and compare against standard LoRA and MoE-LoRA [14]. For the query-aware setting, we evaluate our predictor-based selector, compare it with Expected Attention [3], and further study their combination to examine the complementarity between token-level and sub-token-level query-aware routing. Additional comparison with Quest [18] is reported in the Appendix.

Tasks and metrics. For the query-independent setting, we use continued pretraining on WikiText-103 [15] and report validation perplexity. For the query-aware setting, we use 5-shot MMLU [4] as the main benchmark and report task accuracy together with the corresponding KV budget. We also evaluate long-context behavior using a variable-tracking task from the RULER benchmark [5]. Additional Needle-in-haystack result is included in the Appendix.

4.2 Query-Independent Routing for Language Modeling

We first evaluate the query-independent setting for LoRA adaptation with compression-aware language modeling. The goal is to test whether routed subspace LoRA and value-group routing can preserve language-model quality while reducing KV cost.

Table 1 summarizes the query-independent language-modeling results across Qwen2.5 family. We report standard LoRA, MoE-LoRA, and the query-independent Sub-token method. The KV cache percentage denotes the retained cache size relative to the full-cache baseline.

This comparison should be read from a quality-compression perspective. LoRA and MoE-LoRA are full-cache adaptation baselines, and our Sub-token method evaluates the query-independent design under reduced KV cache. Results show that our Sub-token method improves over both LoRA and MoE-LoRA on all tested Qwen scales while retaining only 75% of the KV cache. This indicates that routed subspace adaptation and routed KV compression play complementary roles: the former improves language-model quality in main optimization, while the latter reduces KV cache cost.

4.3 Query-Aware Value-Group Routing for Downstream KV Compression

In the query-aware setting, the routed object remains the value representation, but the routing score depends on the current query. Instead of assigning a fixed number of value groups per token from the token representation alone, the method scores context-token / value-group pairs using a query-conditioned signal and allocates the budget globally. The resulting number of preserved value groups is therefore token-dependent.

4.3.1 Predictor-based query-aware routing

We use a fixed-split setting, i.e., fixed split layer $L_{split} = 22$, in the main experiments to keep the evaluation protocol consistent across baselines and ablations, so the main comparison reflects differences in routing rather than differences in compression rates. Table 2 reports the results. A further ablation analysis with proportional split is reported separately in the Appendix.

Table 2: Predictor-based selector with fixed split layer $L_{split} = 22$. Here ρ denotes the retained value-group budget, and the corresponding total KV retention is $(1 + \rho)/2$, since only the value cache is compressed. Accuracy retention percentages are shown in parentheses.

Model	Baseline	$\rho = .25$	$\rho = .50$	$\rho = .75$
Qwen2.5-7B	68.94	66.42 (96.3%)	67.92 (98.5%)	68.61 (99.5%)
Qwen2.5-14B	76.71	69.88 (91.1%)	73.96 (96.4%)	75.65 (98.6%)
Qwen2.5-32B	80.76	60.08 (74.4%)	71.45 (88.5%)	78.85 (97.6%)
Qwen2.5-72B	83.26	63.97 (76.8%)	73.97 (88.8%)	80.97 (97.2%)

Table 3: Token-level Expected Attention and sub-token value-group routing across Qwen2.5 model scales on MMLU. Accuracy retention percentages are shown in parentheses.

Model	Method	Tokens kept	V groups kept	Total KV	MMLU
Qwen2.5-7B	Baseline	100%	100%	100.0%	68.94
	EA	75%	100%	75.0%	68.89 (99.9%)
	Sub-token	100%	50%	75.0%	67.92 (98.5%)
	EA	62.5%	100%	62.5%	68.79 (99.8%)
	Sub-token	100%	25%	62.5%	66.42 (96.3%)
	EA + Sub-token	75%	75%	65.6%	68.63 (99.6%)
	EA + Sub-token	75%	50%	56.2%	67.98 (98.6%)
	EA + Sub-token	50%	50%	37.5%	68.32 (99.1%)
Qwen2.5-14B	Baseline	100%	100%	100.0%	76.71
	EA	75%	100%	75.0%	76.61 (99.9%)
	Sub-token	100%	50%	75.0%	76.40 (99.6%)
	EA	62.5%	100%	62.5%	76.63 (99.9%)
	Sub-token	100%	25%	62.5%	76.07 (99.2%)
	EA + Sub-token	75%	75%	65.6%	76.53 (99.8%)
	EA + Sub-token	75%	50%	56.2%	76.39 (99.6%)
	EA + Sub-token	50%	50%	37.5%	76.39 (99.6%)
Qwen2.5-32B	Baseline	100%	100%	100.0%	80.76
	EA	75%	100%	75.0%	80.78 (100.0%)
	Sub-token	100%	50%	75.0%	80.56 (99.8%)
	EA	62.5%	100%	62.5%	80.68 (99.9%)
	Sub-token	100%	25%	62.5%	80.42 (99.6%)
	EA + Sub-token	75%	75%	65.6%	80.76 (100.0%)
	EA + Sub-token	75%	50%	56.2%	80.54 (99.7%)
	EA + Sub-token	50%	50%	37.5%	80.45 (99.6%)
Qwen2.5-72B	Baseline	100%	100%	100.0%	83.26
	EA	75%	100%	75.0%	83.09 (99.8%)
	Sub-token	100%	50%	75.0%	83.00 (99.7%)
	EA	62.5%	100%	62.5%	83.19 (99.9%)
	Sub-token	100%	25%	62.5%	82.92 (99.6%)
	EA + Sub-token	75%	75%	65.6%	83.13 (99.8%)
	EA + Sub-token	75%	50%	56.2%	82.94 (99.6%)
	EA + Sub-token	50%	50%	37.5%	83.10 (99.8%)

The predictor preserves downstream accuracy well across all model sizes, with stronger retention at larger budgets and larger scales. At $\rho = .75$, the compressed model remains close to the baseline across all four Qwen sizes. At lower budgets, the larger models are more robust, which is consistent with the broader trend that larger value representations tolerate more internal compression.

In the Appendix, proportional split scales the split layer with model depth. The results show that part of the fixed-split degradation comes from larger models compressing a larger fraction of their depth.

4.3.2 Relation to token-level query-aware routing

We compare sub-token value-group routing with token-level query-aware routing based on Expected Attention, and also study their combination. Expected Attention selects which tokens survive globally, while value-group routing compresses the surviving tokens internally. This comparison shows

Table 4: Token-level Expected Attention and sub-token value-group routing across Qwen2.5 model scales on variable-tracking long-context. KV retention percentages are shown in parentheses.

Model	Baseline	Sub-token (62.5%)	EA (62.5%)	EA+Sub-token (37.5%)
Qwen2.5-7B	29.5	29.5	29.5	29.5
Qwen2.5-14B	77.0	77.0	77.0	77.0
Qwen2.5-32B	79.0	79.0	79.0	79.0
Qwen2.5-72B	79.5	79.5	79.5	79.5

both how sub-token routing performs relative to a token-level baseline and whether the two are complementary when combined.

Table 3 reports the MMLU results. Two patterns are clear. First, Expected Attention is stronger than sub-token routing alone, especially at smaller scale, which is consistent with token-level selection being a coarser and easier prediction problem. The gap nevertheless narrows with scale, and by 32B and 72B sub-token routing is already very close to the token-level baseline at the same KV budget.

Second, the two methods are complementary. Across Qwen sizes, combining Expected Attention with sub-token routing yields deeper KV compression than either method alone while preserving nearly the same accuracy. At 37.5% total KV, the combined method reaches 68.32 on 7B, corresponding to 99.1% of baseline, and retains 99.6% on 14B and 32B and 99.8% on 72B. This is consistent with larger models containing more redundancy and therefore tolerating more selective preservation.

We also evaluate long-context behavior. Standard needle-in-the-haystack quickly saturates on larger models (14B and above as shown in the Appendix), so we additionally consider a harder variable-tracking task that requires following scattered assignments rather than retrieving a single literal fact. Table 4 gives the result, where compressed methods match the baseline performance with 37.5% total KV. This suggests that the complementarity between token-level and sub-token-level routing extends to harder long-context reasoning.

This complementarity is also efficient computationally. All predictor-based variants, including Expected Attention alone, sub-token routing alone, and the combined method, add essentially no inference overhead relative to baseline. Detailed runtime and memory analysis, together with additional comparison against Quest, are reported in the Appendix.

4.4 Limitations

The results should be interpreted within the scope of the present study. Our methods are developed and evaluated in LoRA-adapted transformers, so it remains open how well the same designs transfer beyond the frozen-backbone adaptation setting. In the query-aware case, sub-token routing does not uniformly exceed token-level baselines in raw accuracy; its value lies in providing a finer compression axis and in combining naturally with token-level selection. The experiments also focus on standard benchmark settings and moderate compression budgets, while longer-context and more aggressive compressed scenarios may reveal different tradeoffs. Currently value-groups are contiguous dimension slices, and are not learned. Semantically meaningful learned groupings might further improve routing quality.

5 Conclusion

We studied sub-token routing as a fine-grained control mechanism for adaptation and KV-cache compression in LoRA-adapted transformers. The key idea is that a relevant token need not be internally uniform, so KV compression need not be an all-or-nothing decision at token level. Based on this view, we introduced query-independent routing for compression-aware language modeling and query-aware routing for downstream-task-preserving KV compression.

The results show that this finer routing is useful in both settings. The query-independent design improves the quality–compression tradeoff for language modeling, while the query-aware design preserves downstream behavior well under reduced KV budgets. We further showed that token-level and sub-token-level query-aware routing are complementary: token-level selection determines which tokens survive globally, while sub-token routing determines how the surviving tokens are compressed internally. Their combination yields deeper KV compression at nearly unchanged task accuracy.

References

- [1] Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., Sanghai, S.: Gqa: Training generalized multi-query transformer models from multi-head checkpoints. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (2023)
- [2] Devoto, A., Jeblick, M., Jégou, S.: Expected attention: Kv cache compression by estimating attention from future queries distribution. *arXiv preprint arXiv:2510.00636* (2025)
- [3] Devoto, A., Jeblick, M., Jégou, S.: Expected attention: Kv cache compression by estimating future attention. *arXiv preprint arXiv:2510.00636* (2025)
- [4] Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., Steinhardt, J.: Measuring massive multitask language understanding. In: *International Conference on Learning Representations (ICLR)* (2021), <https://openreview.net/forum?id=d7KBjmI3GmQ>
- [5] Hsieh, C.P., Sun, S., Krman, S., Acharya, S., Rekesh, D., Jia, F., Ginsburg, B.: RULER: What’s the real context size of your long-context language models? In: *Conference on Language Modeling (COLM)* (2024), <https://arxiv.org/abs/2404.06654>
- [6] Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W.: Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685* (2021)
- [7] Jiang, A.Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D.S., de las Casas, D., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., et al.: Mistral 7B. *arXiv preprint arXiv:2310.06825* (2023), <https://arxiv.org/abs/2310.06825>
- [8] Kumar, A.: Residual vector quantization for kv cache compression in large language model. *arXiv preprint arXiv:2410.15704* (2024)
- [9] Liu, A., Liu, J., Pan, Z., He, Y., Haffari, G., Zhuang, B.: Minicache: Kv cache compression in depth dimension for large language models. In: *Advances in Neural Information Processing Systems* (2024)
- [10] Liu, D., et al.: Query-aware cache selection for efficient llm serving. *arXiv preprint arXiv:2509.12211* (2025)
- [11] Liu, S.Y., Wang, C.Y., Yin, H., Molchanov, P., Wang, Y.C.F., Cheng, K.T., Chen, M.H.: Dora: Weight-decomposed low-rank adaptation. *arXiv preprint arXiv:2402.09353* (2024)
- [12] Liu, Y., Li, H., Cheng, Y., Ray, S., Huang, Y., Zhang, Q., Du, K., Yao, J., Lu, S., Ananthanarayanan, G., Maire, M., Hoffmann, H., Holtzman, A., Jiang, J.: Cachegen: Kv cache compression and streaming for fast large language model serving. In: *Proceedings of the ACM SIGCOMM 2024 Conference* (2024)
- [13] Liu, Z., Luo, J.: Adamole: Adaptive mixture of low-rank adaptation experts. *arXiv preprint arXiv:2405.00361* (2024)
- [14] Luo, T., Lei, J., Lei, F., Liu, W., He, S., Zhao, J., Liu, K.: Moelora: Contrastive learning guided mixture of experts on parameter-efficient fine-tuning for large language models. *arXiv preprint arXiv:2402.12851* (2024)
- [15] Merity, S., Xiong, C., Bradbury, J., Socher, R.: Pointer sentinel mixture models. In: *International Conference on Learning Representations (ICLR)* (2017), <https://openreview.net/forum?id=Byj72udxe>
- [16] Qwen Team: Qwen2.5 technical report (2024), <https://qwenlm.github.io/blog/qwen2.5/>
- [17] Shazeer, N.: Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150* (2019)
- [18] Tang, J., Zhao, Y., Zhu, K., Xiao, G., Kasikci, B., Han, S.: Quest: Query-aware sparsity for efficient long-context llm inference. *arXiv preprint arXiv:2406.10774* (2024)

- [19] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. In: Advances in Neural Information Processing Systems. vol. 30 (2017)
- [20] Wu, X., Huang, S., Ye, Y., Xia, F., Stoyanov, V., Roth, D.: Mixture of lora experts. In: International Conference on Learning Representations (2024)
- [21] Xiao, G., Tian, Y., Chen, B., Han, S., Lewis, M.: Efficient streaming language models with attention sinks. In: International Conference on Learning Representations (2024)
- [22] Zhang, Q., Chen, M., Bukharin, A., Karampatziakis, N., He, P., Cheng, Y., Chen, W., Zhao, T.: Adalora: Adaptive budget allocation for parameter-efficient fine-tuning. In: International Conference on Learning Representations (2023)
- [23] Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Ré, C., Barrett, C., Wang, Z., Chen, B.: H₂O: Heavy-hitter oracle for efficient generative inference of large language models. arXiv preprint arXiv:2306.14048 (2023)

A Additional Method Details

A.1 Optimization Objectives

Query-independent routing. The query-independent model contains two components with different roles: routed subspace LoRA on the main language-model path and value-group routing on the cache side. Let $\mathcal{L}_{\text{LM}}$ denote the standard language-modeling loss and let $\mathcal{L}_{\text{aux}}$ denote an auxiliary loss on the cache side. The overall objective is

$$\mathcal{L} = \mathcal{L}_{\text{LM}} + \lambda \mathcal{L}_{\text{aux}}. \quad (14)$$

The language-model objective updates the routed LoRA modules. The auxiliary objective updates only the routing and reconstruction modules on the cache side. Specifically,

$$\mathcal{L}_{\text{aux}} = \ell(\hat{V}, V), \quad (15)$$

where $\ell(\cdot, \cdot)$ is a reconstruction loss between the original value tensor V and its reconstruction $\hat{V}$.

Query-aware routing. The predictor-based query-aware model uses a learned scorer to approximate token-group relevance scores. For each context token j , let $h_j^{(L_{\text{split}})}$ denote the hidden state at the split layer and let $s_{j,:} \in \mathbb{R}^S$ denote the target scores over value groups. The predictor outputs $\hat{s}_{j,:}$ and is trained by minimizing

$$\mathcal{L}_{\text{pred}} = \sum_j \ell(\hat{s}_{j,:}, s_{j,:}), \quad (16)$$

where $\ell(\cdot, \cdot)$ is a regression loss.

A.2 Optimization and Training Details

This subsection summarizes the optimization settings used for the query-independent and query-aware components.

Query-independent training. The full query-independent model combines routed subspace LoRA with value-group routing on the KV path. For routed LoRA, the number of routed subspaces is $S = 4$, with top- K routing $K = 2$, LoRA rank $r = 4$, LoRA scaling $\alpha = 8.0$, and dropout 0.05.

For value-group routing on the KV path, we use $S = 4$ value groups and retain top- $K = 2$ groups, corresponding to 50% value retention and 75% total KV retention. The auxiliary compression loss weight is 0.1, and the load-balance weight is 0.01. The routing is applied to the self-attention modules.

Optimization uses AdamW with learning rate 5×10^{-4} , weight decay 0.01, and betas (0.9, 0.999). The learning-rate schedule uses linear warmup followed by cosine decay to zero. Warmup lasts 1000 steps for 7B, 14B, and Mistral-7B, and 500 steps for 32B and 72B. Training is performed on the

WikiText-103-raw-v1 train split with sequence length 512, per-device batch size 1, and gradient accumulation 8, giving an effective batch size of 8. Validation uses at most 200 samples from the WikiText-103 validation split. We train 7B, 14B, and Mistral-7B for 50,000 steps, and 32B and 72B for 20,000 steps. Evaluation is performed every 5,000 steps for 7B and 14B, and every 2,000 steps for 32B and 72B. The best checkpoint is selected by validation perplexity.

MoE-LoRA baseline. For the MoE-LoRA baseline, we use $E = 4$ experts, top- $K = 2$ routing, LoRA rank $r = 8$, LoRA scaling $\alpha = 16.0$, and dropout 0.05. All other hyperparameters, including optimizer, schedule, data, training steps, and evaluation protocol, are the same as in the query-independent setting above.

Query-aware predictor training. The predictor for sub-token query-aware routing is a two-layer MLP of the form

$$d_{\text{model}} \rightarrow 256 \rightarrow S,$$

with GELU activation, where $S = 4$ for value-group prediction. The predictor size is approximately 920K parameters for 7B, 1.05M for 14B, 1.3M for 32B, and 2.1M for 72B.

Training data is collected from 500 WikiText-103-raw-v1 training documents, with maximum sequence length 512. Optimization uses AdamW with learning rate 10^{-3} and weight decay 10^{-4} . We train for 50 epochs over the collected data, with batch size 1024 tokens and no warmup. The loss is mean-squared error between predicted and target importance scores, with targets normalized per group to zero mean and unit var. Training takes approximately 5 minutes per model on a single H100.

Expected Attention predictor. The Expected Attention predictor uses the same training pipeline as the value-group predictor, but with a scalar prediction head of the form

$$d_{\text{model}} \rightarrow 256 \rightarrow 1.$$

It predicts token-level received attention mass rather than a per-group importance vector. All other training details are unchanged.

Split-layer settings. For the proportional-split setting, the split layer is chosen so that approximately the deepest 22% of layers are compressed. The resulting split layers are 22 for Qwen2.5-7B, 38 for Qwen2.5-14B, 50 for Qwen2.5-32B, 62 for Qwen2.5-72B, and 25 for Mistral-7B.

Evaluation procedures. For MMLU, we use 5-shot in-context learning over 57 subjects and 14,042 test questions, with maximum sequence length 2048. Accuracy is computed by taking the maximum logit over the answer options A/B/C/D. For needle-in-the-haystack evaluation, we use 200 examples per configuration. Each example contains a synthetic fact of the form “The magic number is X ,” where X is a random digit from 1 to 9, inserted at a random position in the context. The model generates up to 10 tokens, and retrieval is counted as correct if the target digit appears. For perplexity, we evaluate on 200 samples from the WikiText-103 validation split and report the exponential of the average cross-entropy loss. For inference profiling, we use a single H100 NVL, 50 forward passes per mode with warmup excluded, prompt length around 400 tokens, and batch size 1. We report wall-clock latency and peak GPU memory.

A.3 Asset Sources, Versions, and Licenses

This subsection lists the main external assets used in the paper, including pretrained models, datasets, and software libraries. All assets are used in accordance with their stated licenses or terms of use. The experiments use publicly available pretrained models and standard benchmark datasets for research evaluation only. We do not redistribute any pretrained models or datasets.

Pretrained models. Table 5 lists the pretrained model checkpoints used in the experiments.

Datasets. Table 6 lists the datasets used in training and evaluation.

WikiText-103 is derived from Wikipedia articles, which are licensed under CC-BY-SA 3.0. In this work, WikiText-103-raw-v1 is used for continued pretraining in the query-independent setting and for perplexity evaluation on the validation split. MMLU is used for 5-shot multiple-choice evaluation in the query-aware setting.

Table 5: Pretrained models used in the experiments.

Asset	Version	License	URL
Qwen2.5-7B	—	Apache 2.0	https://huggingface.co/Qwen/Qwen2.5-7B
Qwen2.5-14B	—	Apache 2.0	https://huggingface.co/Qwen/Qwen2.5-14B
Qwen2.5-32B	—	Apache 2.0	https://huggingface.co/Qwen/Qwen2.5-32B
Qwen2.5-72B	—	Apache 2.0	https://huggingface.co/Qwen/Qwen2.5-72B
Mistral-7B-v0.3	v0.3	Apache 2.0	https://huggingface.co/mistralai/Mistral-7B-v0.3

Table 6: Datasets used in the experiments.

Asset	License	URL
WikiText-103-raw-v1	CC-BY-SA 3.0	https://huggingface.co/datasets/Salesforce/wikitext
MMLU	MIT	https://huggingface.co/datasets/cais/mmlu

Software libraries. Table 7 lists the main software libraries used in the experiments.

Table 7: Software libraries used in the experiments.

Asset	Version	URL
PyTorch	2.5.1	https://pytorch.org
Transformers	5.5.3	https://github.com/huggingface/transformers
PEFT	0.18.1	https://github.com/huggingface/peft
BitsAndBytes	0.49.2	https://github.com/bitsandbytes-foundation/bitsandbytes
Datasets	4.8.4	https://github.com/huggingface/datasets

Intended use. All pretrained models are used for research evaluation only, specifically for studying KV-cache compression and routing methods. No pretrained model or dataset is redistributed as part of this work. All datasets are used within the terms of their respective licenses.

B Additional Query-Independent Results

B.1 Cross-Family Query-Independent Language-Modeling Results

To verify that the query-independent design is not specific to the Qwen architecture, we additionally evaluate it on Mistral-7B. Table 8 shows that our method also improves over LoRA and MoE-LoRA on this second model family while reducing KV cache size. This supports the view that the query-independent routing mechanism is not tied to a particular backbone family.

C Additional Query-Aware Results

C.1 Proportional-Split Predictor Across Model Scales

The main text uses a fixed split setting, in which the split layer is set to $L_{split} = 22$ for all model sizes. This keeps the evaluation protocol consistent across the main query-aware experiments, but it also means that different models are compressed over different fractions of their depth. In particular, the same absolute split is relatively late in a smaller model and relatively early in a larger one, so larger models must rely on the routed value representation over many more downstream layers.

To examine this effect, we additionally evaluate a proportional-split setting, in which the split layer is scaled with model depth so that approximately the same fraction of late layers is compressed across models. This analysis is intended to separate the effect of the routing method itself from the effect of compressing different relative portions of the network under the fixed $L_{split} = 22$ setting.

Table 9 reports the results. When split is chosen proportionally, the predictor becomes nearly lossless across all four Qwen sizes. In particular, at $\rho = .25$, corresponding to 62.5% total KV retention,

Table 8: Cross-family query-independent language-modeling results at 7B scale. LoRA and MoE-LoRA are full-cache baselines. KV retention percentages are shown in parentheses.

Model	LoRA	MoE-LoRA	Sub-Token
Qwen2.5-7B	35.08 (100%)	33.72 (100%)	31.41 (75%)
Mistral-7B	28.71 (100%)	26.16 (100%)	24.81 (75%)

Table 9: Predictor-based selector with proportional split layer. The split layer is chosen so that approximately 22% of the deepest layers are compressed across models. Here ρ denotes the retained value-group budget, and the corresponding total KV retention is $(1 + \rho)/2$, since only the value cache is compressed. Accuracy retention percentages are shown in parentheses.

Model	Split	Layers	Baseline	$\rho = .25$	$\rho = .50$	$\rho = .75$
Qwen2.5-7B	22	5/28	68.94	66.42 (96.3%)	67.92 (98.5%)	68.61 (99.5%)
Qwen2.5-14B	38	10/48	76.71	76.07 (99.2%)	76.40 (99.6%)	76.53 (99.8%)
Qwen2.5-32B	50	14/64	80.76	80.42 (99.6%)	80.56 (99.8%)	80.73 (99.9%)
Qwen2.5-72B	62	18/80	83.26	82.92 (99.6%)	83.00 (99.7%)	83.14 (99.9%)

accuracy retention reaches 99.2% on 14B, 99.6% on 32B, and 99.6% on 72B. This suggests that a substantial part of the degradation observed under the fixed-split setting comes from the mismatch in compressed depth across scales rather than from the predictor-based routing mechanism itself.

C.2 Cross-Family Query-Aware Results on Mistral-7B

To test whether the query-aware findings depend strongly on the Qwen family, we additionally evaluate the combined token-level and sub-token-level design on Mistral-7B. Table 10 gives the results. From the table, results support the same conclusion as in the Qwen models: token-level Expected Attention and sub-token value-group routing act on complementary compression axes. Even at aggressive budgets, their combination remains nearly lossless. In particular, the combined method retains 99.9–100.0% of baseline accuracy at total KV budgets between 37.5% and 65.6%, providing a cross-family check for the complementarity result in the main text.

Table 10: Cross-family query-aware results on Mistral-7B. Expected Attention performs token-level query-aware selection, while sub-token routing compresses internal value groups. Accuracy retention percentages are shown in parentheses.

Method	Tokens kept	V groups kept	Total KV	MMLU	Retention
Baseline	100%	100%	100.0%	59.39	100.0%
EA + Sub-token	75%	75%	65.6%	59.29	99.8%
EA + Sub-token	75%	50%	56.2%	59.39	100.0%
EA + Sub-token	50%	50%	37.5%	59.34	99.9%

C.3 Fixed- K Query-Independent Routing versus Predictor-Based Query-Aware Routing

Table 11 compares fixed- K query-independent routing with predictor-based query-aware routing on Qwen2.5-7B under matched budgets. The purpose of this comparison is to isolate the effect of adaptive, query-conditioned allocation. In the fixed- K setting, every token is forced to keep the same number of value groups. In the query-aware setting, the predictor allocates the same overall budget unevenly across tokens according to query-conditioned relevance.

The difference is consistent at all three budgets. This aligns with the motivation for query-aware routing: once the downstream query is known, the useful information is not distributed uniformly across tokens, and a fixed per-token allocation becomes unnecessarily restrictive. The predictor-based method instead concentrates preservation on the tokens that matter most for the current query, which yields much stronger downstream accuracy under the same overall budget.

Table 11: Fixed- K query-independent routing versus query-aware routing on Qwen2.5-7B.

Method	Budget	MMLU	Retention
Fixed $K = 3$	75%	62.59	90.8%
Query-aware predictor	75%	68.61	99.5%
Fixed $K = 2$	50%	29.13	42.2%
Query-aware predictor	50%	67.92	98.5%
Fixed $K = 1$	25%	26.71	38.7%
Query-aware predictor	25%	66.42	96.3%

C.4 Comparison with Quest

The main text compares our predictor-based method with Expected Attention as the primary one-pass token-level baseline. Here we additionally report Quest, which is a query-aware baseline based on a diagnostic stage and therefore follows a different inference path. Table 12 gives the results. The purpose of this comparison is to show that, although Quest operates at a coarser routing granularity and uses a two-pass procedure, its accuracy converges toward the same range as our method at larger model scales.

Table 12: Comparison with Quest across matched KV budgets. Here ρ denotes the retained value-group budget, and the corresponding total KV retention is $(1 + \rho)/2$, since only the value cache is compressed. Accuracy retention percentages are shown in parentheses. Quest uses an additional diagnostic pass, while our method is one-pass.

Model	Budget	Quest	Ours
Qwen2.5-7B	$\rho = .25$	68.42 (99.2%)	66.42 (96.3%)
	$\rho = .50$	68.72 (99.7%)	67.92 (98.5%)
	$\rho = .75$	68.91 (100.0%)	68.61 (99.5%)
Qwen2.5-14B	$\rho = .25$	76.74 (100.0%)	76.07 (99.2%)
	$\rho = .50$	76.79 (100.1%)	76.40 (99.6%)
	$\rho = .75$	76.67 (100.0%)	76.53 (99.8%)
Qwen2.5-32B	$\rho = .25$	80.38 (99.5%)	80.42 (99.6%)
	$\rho = .50$	80.47 (99.6%)	80.56 (99.8%)
	$\rho = .75$	80.68 (99.9%)	80.73 (100.0%)
Qwen2.5-72B	$\rho = .25$	83.01 (99.7%)	82.92 (99.6%)
	$\rho = .50$	83.14 (99.9%)	83.00 (99.7%)
	$\rho = .75$	83.23 (100.0%)	83.14 (99.9%)

C.5 Needle-in-the-Haystack Evaluation

We include needle-in-the-haystack retrieval results as a complementary long-context evaluation for the query-aware setting. Table 13 reports the results across Qwen2.5 model scales. The main pattern is consistent with the MMLU results in the main text. Query-aware compression remains stable under substantial KV reduction, and the combined token-level and sub-token-level design preserves retrieval well. At 37.5% total KV, the combined method retains 84.5% accuracy on 7B and 100.0% on 14B, 32B and 72B.

A natural explanation is that larger models have greater representational redundancy and stronger capacity to recover from partial compression of the value path. Needle retrieval is a relatively structured long-context task, and once the model is large enough, both token-level selection and sub-token routing appear sufficient to preserve the information needed for exact retrieval. As a result, the distinction between compression settings is most visible at 7B, while 14B and above remain effectively lossless even at aggressive budgets.

Table 13: Needle-in-the-haystack results across model scales. Context length is 2500 tokens, with 200 examples and a 10-token generation window.

Model	Method	Tokens kept	V groups kept	Total KV	Needle Acc
Qwen2.5-7B	Baseline	100%	100%	100.0%	85.0%
	Expected Attention	62.5%	100%	62.5%	85.0%
	Expected Attention	75%	100%	75.0%	85.0%
	Sub-token	100%	25%	62.5%	84.0%
	Sub-token	100%	50%	75.0%	84.0%
	EA + Sub-token	75%	75%	65.6%	85.0%
	EA + Sub-token	75%	50%	56.2%	85.0%
	EA + Sub-token	50%	50%	37.5%	84.5%
Qwen2.5-14B / 32B / 72B	Baseline	100%	100%	100.0%	100.0%
	Expected Attention	62.5%	100%	62.5%	100.0%
	Expected Attention	75%	100%	75.0%	100.0%
	Sub-token	100%	25%	62.5%	100.0%
	Sub-token	100%	50%	75.0%	100.0%
	EA + Sub-token	75%	75%	65.6%	100.0%
	EA + Sub-token	75%	50%	56.2%	100.0%
	EA + Sub-token	50%	50%	37.5%	100.0%

C.6 Predictor-Based Runtime Profiling

Table 14 reports single-GPU profiling for the main predictor-based methods. All one-pass predictor-based variants, including sub-token routing, Expected Attention, and their combination, add essentially no latency overhead relative to the baseline. We also report peak GPU memory for the same runs. Together, these results support the main-text claim that the one-pass query-aware variants improve the quality–compression tradeoff without materially changing the inference costs.

Table 14: Inference profiling on a single H100 NVL with ~ 400 -token prompts. Latency is reported in milliseconds and peak memory in MB.

Mode	Latency (ms) / Peak memory (MB)			
	Qwen2.5-7B	Qwen2.5-14B	Qwen2.5-32B	Qwen2.5-72B
Baseline	36.5 / 5,511	64.2 / 9,748	111.4 / 18,742	233.2 / 39,943
Sub-token	36.4 / 5,500	62.3 / 9,708	112.7 / 18,688	234.5 / 39,876
EA	36.3 / 5,500	61.8 / 9,709	113.7 / 18,688	234.7 / 39,876
EA + Sub-token	36.4 / 5,500	61.9 / 9,709	114.0 / 18,688	236.4 / 39,877